Academic Year: 2026-2027
Semester: III
Class / Branch: SE IT
Subject: ADSAL
Name of Instructor: Ms. Sneha Dalvi

Name of Student:Nishant Suryarao
Student ID:26204001
Date of Performance:22/09/2026
Date of Submission:22/09/2026

Experiment No.4

Aim: Implement a solution for a 0-1 knapsack problem using dynamic programming.

Code:

```
#include <stdio.h>
// Function to get the maximum of two
numbers int max(int a, int b) {  return (a > b) ?
a : b;
}
// Knapsack function int knapsack(int W, int wt[],
int val[], int n) {int i, w;  int dp[n + 1][W + 1];
 // Build table dp[][] in bottom-up manner  for (i = 0; i <= n;
i++) {  for (w = 0; w <= W; w++) {  if (i == 0 || w == 0)  dp[i][w]
= 0;  else if (wt[i - 1] <= w)  dp[i][w] = max(val[i - 1] + dp[i -
1][w - wt[i - 1]], dp[i - 1][w]);  else  dp[i][w] = dp[i - 1][w];
 } }  return
dp[n][W]; }
int main() {
int n, W;
```

```
printf("Enter number of items: ");
scanf("%d", &n);  int val[n], wt[n];
printf("Enter values of the
items:\n");  for (int i = 0; i < n; i++) {
printf("Value of item %d: ", i + 1);
scanf("%d", &val[i]);
}printf("Enter weights of the
items:\n");  for (int i = 0; i < n; i++) {
printf("Weight of item %d: ", i + 1);
scanf("%d", &wt[i]);
}  printf("Enter capacity of knapsack: ");
scanf("%d", &W);  int result = knapsack(W, wt, val,
n);  printf("Maximum value in knapsack: %d\n",
result);  return 0;
}
```

Output Screen Shots:

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$ gcc -o Exp4 Exp4.c
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$ ./Exp4
Enter number of items: 3
Enter values of the items:
Value of item 1: 60
Value of item 2: 100
Value of item 3: 120
Enter weights of the items:
Weight of item 1: 10
Weight of item 2: 20
Weight of item 3: 30
Enter capacity of knapsack: 50
Maximum value in knapsack: 220
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$
```

Conclusion: Successfully implemented the 0-1 Knapsack problem using Dynamic Programming. The experiment helped us understand how DP avoids

repeated calculations and finds the maximum value within the given capacity. For the given data, the maximum value obtained is 220 by selecting Items 2 and 3.

Experiment 4: 0/1 Knapsack Using Dynamic Programming

Definition: The 0/1 Knapsack problem is an optimization problem in which each item has a weight and a value. The objective is to maximize the total value without exceeding the knapsack capacity. Each item can either be selected or rejected.

Algorithm:

1. Start.
2. Read the number of items, their values, weights, and knapsack capacity \(W\).
3. Create a DP table dp[n+1][W+1].
4. Initialize the first row and column with zero.
5. For each item and capacity, check whether the item can fit.
6. If it fits, choose the maximum of including or excluding the item.
7. Otherwise, copy the value from the previous row.
8. Display dp[n][W] as the maximum value.
9. Stop.

Conclusion: Thus, the 0/1 Knapsack problem was successfully implemented using Dynamic Programming to find the maximum value within the given capacity.

Complexity: Time \(O(nW)\), Space \(O(nW)\).